

SECURITY AND PERFORMANCE CONSIDERATIONS

Learning Objectives:

Understand security risks and apply basic protection techniques in integrated systems

Differentiate authentication and authorization, including the use of Role-Based Access Control

Analyze system performance and suggest optimization methods

Explain scalability and how systems handle growth

Evaluate reliability and methods to ensure system availability

Recognize trade-offs between security, performance, and scalability

Security and Performance Considerations in Integrated Systems

Integrated systems (multiple applications, databases, and services working together) introduce complex risks and performance challenges. A weakness in one component can affect the entire ecosystem.

1. Security in Integrated Systems

Refers to protecting data, services, and communication flows across interconnected components. These systems often follow architectures like APIs, microservices, or event-driven systems — making them powerful, but also more vulnerable.

Common Risks

- Data interception (e.g., APIs exposed over the internet)
- Unauthorized system access
- Data leakage between systems
- Weak integration points (APIs, middleware)

Enterprise Security

Authentication · Access Control

Network Security · Encryption

Infrastructure Security

Audit & Monitoring · System Security

Security and Performance Considerations in Integrated Systems

Data Interception

Data is captured while traveling between systems.

Why: No encryption · Public networks · Poor API security

Impact: Stolen credentials · Exposure of sensitive data

Prevention: Use HTTPS/TLS · Encrypt sensitive data · Secure API communication

Unauthorized System Access

Someone accesses the system without proper permission.

Causes: Weak passwords · No auth controls · Poor access validation

Impact: Data manipulation or deletion · System takeover

Prevention: Strong auth (MFA) · Proper authorization · Session and token management

Data Leakage Between Systems

Data is exposed unintentionally when systems exchange information.

Causes: Poor API design · Lack of data filtering · Over-permissioned systems

Impact: Privacy violations · Legal consequences · Loss of trust

Prevention: Data minimization · Strict access control · Mask/encrypt sensitive fields

Weak Integration Points (APIs, Middleware)

The connections between systems are not secure.

Causes: No input validation · Missing authentication · Poor API management

Impact: Full system compromise · Data breaches across multiple systems

Prevention: Secure APIs (tokens, auth) · Validate all inputs · API gateways & rate limiting

Security and Performance Considerations in Integrated Systems

Security Measures

Encryption

Data in transit → HTTPS, TLS

Data at rest → Database encryption

API Security

API gateways

Rate limiting

Token validation

Network Security

Firewalls, VPNs, segmentation

Logging & Monitoring

Detect anomalies and intrusions

Security and Performance Considerations in Integrated Systems

4-Layer Defense Model

1

Protect the Data

→ Encryption

2

Protect the Access

→ API Security

3

Protect the Network

→ Firewalls, VPNs

4

Watch Everything

→ Logging & Monitoring

Security measures ensure: Data is safe while moving and stored · Access is controlled and verified · Networks protected and segmented · Threats detected in real-time

2. Authentication and Authorization

Authentication — Who are you?

Verifies the identity of a user or system.

- Username & password
- Biometrics
- Multi-Factor Authentication (MFA)

Authorization — What can you do?

Determines access level after authentication.

- Admin vs. User permissions
- Read-only vs. full access

Common Models

RBAC

Role-Based Access Control
Access based on roles
(e.g., Admin, Operator)

ABAC

Attribute-Based Access
Control
Access based on user,
environment & resource
attributes

JWT

Token-Based
Authentication
JSON Web Tokens issued
after login

OAuth/SSO

Single Sign-On
Log in once, access
multiple systems

2. Authentication and Authorization — RBAC

Role-Based Access Control (RBAC)

Access based on roles assigned to users (e.g., Admin, Operator).

Example: Student Information System

Admin

Can edit all records

Instructor

Can input grades

Student

Can only view grades

Advantages

- Simple to implement
- Easy to manage for small–medium systems

Limitation

- Not flexible for complex conditions (e.g., "only allow access during office hours")

2. Authentication and Authorization — ABAC

Attribute-Based Access Control (ABAC)

Access depends on attributes, not just roles.

User

Role, Department

Resource

File type, Sensitivity

Environment

Time, Location, Device

Example: Access is granted ONLY if:

User = Instructor AND Time = Working Hours AND Location = Inside Campus

Advantages

- Highly flexible
- Fine-grained control

Limitation

- More complex to design and manage

2. Authentication and Authorization — JWT & OAuth/SSO

Token-Based Authentication (JWT)

After login, the system issues a token instead of repeatedly asking for credentials.

How it works:

1. User logs in
2. Server generates a JWT
3. Token is sent with every request
4. Server validates the token

Advantages: Fast and scalable · Works well with APIs and mobile apps

Risks: Token theft if not secured · Must use HTTPS and expiration time

OAuth 2.0 / Single Sign-On (SSO)

Allows users to log in once and access multiple systems.

How it works:

- Log in via a trusted provider (e.g., Google)
- Other apps trust that login

Example: Log in with Google → access email, drive, and classroom (one login, multiple services)

Advantages: Convenient for users · Centralized authentication

Risks: If one account is compromised → all connected systems affected

2. Authentication and Authorization — Model Comparison

Model	Focus	Best Use Case
RBAC	Roles	Simple Systems
ABAC	Attributes	Complex, dynamic systems
JWT	Authentication	APIs, web apps
OAuth/SSO	Federated Login	Multiple systems

Modern systems often combine these models:

RBAC or ABAC → for authorization

JWT → for authentication sessions

OAuth/SSO → for cross-system login

3. Performance, Scalability, and Reliability

Performance

Measures how fast the system responds.

Factors:

- Network latency
- Database queries
- API response time
- System load

Optimization:

- Caching (Redis)
- Efficient queries
- Load balancing

Scalability

Ability to handle increasing users or data.

Types:

- Vertical Scaling
Upgrading hardware
- Horizontal Scaling
Adding more servers



Modern systems prefer horizontal scaling (cloud-based)

Reliability

Ensures system availability and fault tolerance.

Strategies:

- Redundancy (backup systems)
- Failover mechanisms
- Data replication
- Monitoring & alerting

Common Issues:

- System downtime
- Bottlenecks
- Single points of failure

3. Performance, Scalability, and Reliability — Trade-offs

Balancing the Three Pillars



Security ↑

Trade-off:

May reduce performance
(e.g., encryption overhead)



Performance ↑

Trade-off:

May weaken security if
shortcuts are used



Scalability ↑

Trade-off:

Requires more complex
system design

Bottom Line



Security

ensures systems are protected across all integration points



Authentication & Authorization

control access and verify identity



Performance, Scalability, and Reliability

ensure systems run efficiently and continuously